

# High-Concurrency Booking Engine

## BookMyShow-style MySQL database solution

### HIGH-CONCURRENCY BOOKING ENGINE

BookMyShow-style MySQL database solution

#### ENTITIES

users  
theatres  
screens  
seats  
movies  
shows  
show\_seats  
bookings  
booking\_items  
payments  
payment\_webhook\_events

#### HIGH-CONCURRENCY DESIGN

show\_seats stores one physical seat for one show. UNIQUE(show\_id, seat\_id) prevents duplicate inventory. Booking should use an InnoDB transaction and SELECT ... FOR UPDATE on the requested seat rows. The transaction checks availability, changes seats to HELD, creates booking records, and commits. A competing transaction waits and then observes the updated state.

#### TIMED HOLDS

A held seat stores status=HELD and hold\_expires\_at. An expiry worker releases expired holds. Redis can be used as a fast distributed hold/expiry layer in a production architecture.

#### IDEMPOTENT PAYMENT WEBHOOKS

payment\_webhook\_events.provider\_event\_id is UNIQUE. A repeated provider event therefore cannot be recorded twice.

#### LOCKING

Pessimistic locking: SELECT ... FOR UPDATE. It is straightforward for hot seats, but competing requests can wait.

Optimistic locking: use the version column and update only when the expected version still matches.

#### NORMALIZATION

1NF: atomic attributes and no repeating groups.

2NF: non-key attributes depend on the complete key.

3NF: independent facts are separated and transitive dependencies avoided.

BCNF: relevant determinants are candidate keys, supported by primary/unique constraints.

#### P2 QUERY

```
SELECT t.theatre_name, m.title AS movie_title, s.show_date,  
s.start_time, s.end_time, sc.screen_name  
FROM shows s  
JOIN screens sc ON sc.screen_id=s.screen_id  
JOIN theatres t ON t.theatre_id=sc.theatre_id  
JOIN movies m ON m.movie_id=s.movie_id  
WHERE t.theatre_id=1 AND s.show_date='2026-10-01'  
ORDER BY s.start_time;
```

FILES  
sql/schema.sql  
README.md  
DESIGN.md  
DEMO.md